



Setup Wizard Guide

*A Detailed, Screen-by-Screen Path from Delphi Source to Offline
Language Packs*



Version 1.0

Last changed: August 30, 2026

Windows • Delphi VCL and FireMonkey • Win32 and Win64

Table of Contents

1. Purpose, Audience, and Final Result	1
2. Before Opening the Wizard	2
3. Wizard Navigation, Keyboard, and Safety	3
4. Step 1 - Welcome	5
5. Step 2 - Delphi Project	6
6. Step 3 - Deployment	8
7. Step 4 - Languages	10
8. Step 5 - Translation Service	11
9. Step 6 - Scan Project	13
10. Step 7 - Review and Authorize	15
11. Localization Review During Final Processing	16
12. Step 8 - Process and Finish	17
13. Component Kit and dependencies Folder	18
14. Delphi Compiler Search Path	19
15. Files and Folders Created or Used	20
16. Build and Deployment Procedure	21
17. Complete Runtime Verification	22
18. Updating, Resuming, and Adding Languages	22
19. Failure Recovery and Troubleshooting	23
20. Security and Privacy	24
21. Printable Completion Checklist	25
22. Quick Reference	26

1. Purpose, Audience, and Final Result

This guide is the complete operating procedure for the eight-step Translation Setup Wizard. It is intended for a Delphi developer who is localizing a VCL or FireMonkey application for the first time, repeating a translation after source changes, or adding another language to an existing application.

The Wizard reads the selected Delphi project, creates or merges a development catalog, translates only eligible unresolved material, opens Localization Review, validates the result, exports a canonical English source pack and translated pack, generates a component integration kit, and deploys packs to detected and authorized application folders. The target Pascal, DFM, FMX, DPR, and DPROJ files remain unchanged.

The deployed application remains offline. DeepL or Google is contacted only by the Studio on the developer computer. The target application reads validated local JSON packs and never receives the provider API key.

Recommended safety boundary. Use a pristine backup and a separate test copy of the target project. A successful Wizard run is reversible because it writes Studio workspace/export artifacts and deployed packs, not application source or project files.

Completion product	What it is	Where it is used
Development catalog	The full editable key/source/translation/context/review model.	Studio workspace under %LOCALAPPDATA%.
Canonical source pack	The validated English or other authored-source runtime pack.	Beside every target EXE under Localization\Languages.
Translated pack	The validated compact target-locale runtime pack.	Loaded by the VCL/FMX runtime manager.
Localization Review package	HTML review, findings, glossary, layout proposals, and decisions.	Studio export\localization-review.
Component integration kit	Complete framework-appropriate runtime/component source, packs, manifest, README, deployment script, and report.	Studio export\component-integration.

Completion product	What it is	Where it is used
Development catalog	The full editable key/source/translation/context/review model.	Studio workspace under %LOCALAPPDATA%.
Safety ZIP	Timestamped pre-processing backup of the selected project.	Reported in Wizard progress and completion report.

2. Before Opening the Wizard

2.1 Obtain and build the Studio

1. Download or clone the complete repository from <https://github.com/tmartindub/DelphiAppTranslationStudio>. Do not download isolated PAS, DPK, BPL, or JSON files.
2. Extract it to a short writable path, for example C:\New Delphi Projects\Delphi App Translation.
3. Open DelphiAppTranslationStudio.dproj in RAD Studio 13 Florence and build Win32 Release for the simplest first run.
4. The verified RAD Studio environment is C:\Program Files (x86)\Embarcadero\Studio\37.0\bin\rvars.bat and the Win32 compiler is dcc32.exe in the same folder.
5. Run bin\Win32\Release\DelphiAppTranslationStudio.exe from the repository copy you just built.

2.2 Prepare the target application

1. Create a pristine backup and a separate test copy. Build and briefly run the test copy before localization work begins.
2. Build the DAT runtime and design packages supplied with the Studio. In RAD Studio choose Component > Install Packages > Add and install the matching Win32 Release design BPL.
3. Open the target primary form in the Form Designer. Place one TDATAVCLLanguageManager or TDATAFMXLanguageManager and one matching language combo box.
4. In Object Inspector set ApplicationId to the exact Delphi project name, LanguagesFolder to Localization\Languages, SourceLanguage to the authored locale, and connect the combo box LanguageManager property.
5. Add and position any visible Language label or menu item in the designer. Choose File > Save All so the first scan sees every intended static string.

6. Close the target project before Wizard final processing. The Wizard verifies this explicitly at Review and Authorize.

Designer-first rule. The supplied manager and selector are normal designer components. Configure them in Object Inspector. The Wizard does not inject runtime UI construction or silently rewrite the form.

2.3 Provider and network readiness

- DeepL is the recommended default. Use a DeepL API Free or API Pro key, not merely a consumer Translator subscription.
- Google Cloud Translation Basic v2 is supported when the API is enabled, billing/quota are valid, and the key restrictions permit this developer computer.
- A stored key is kept in Windows Credential Manager. A session-only key disappears when the Studio closes.
- Test the provider connection before final processing. Do not treat a timeout, 401/403, or 429 result as a successful setup.

3. Wizard Navigation, Keyboard, and Safety

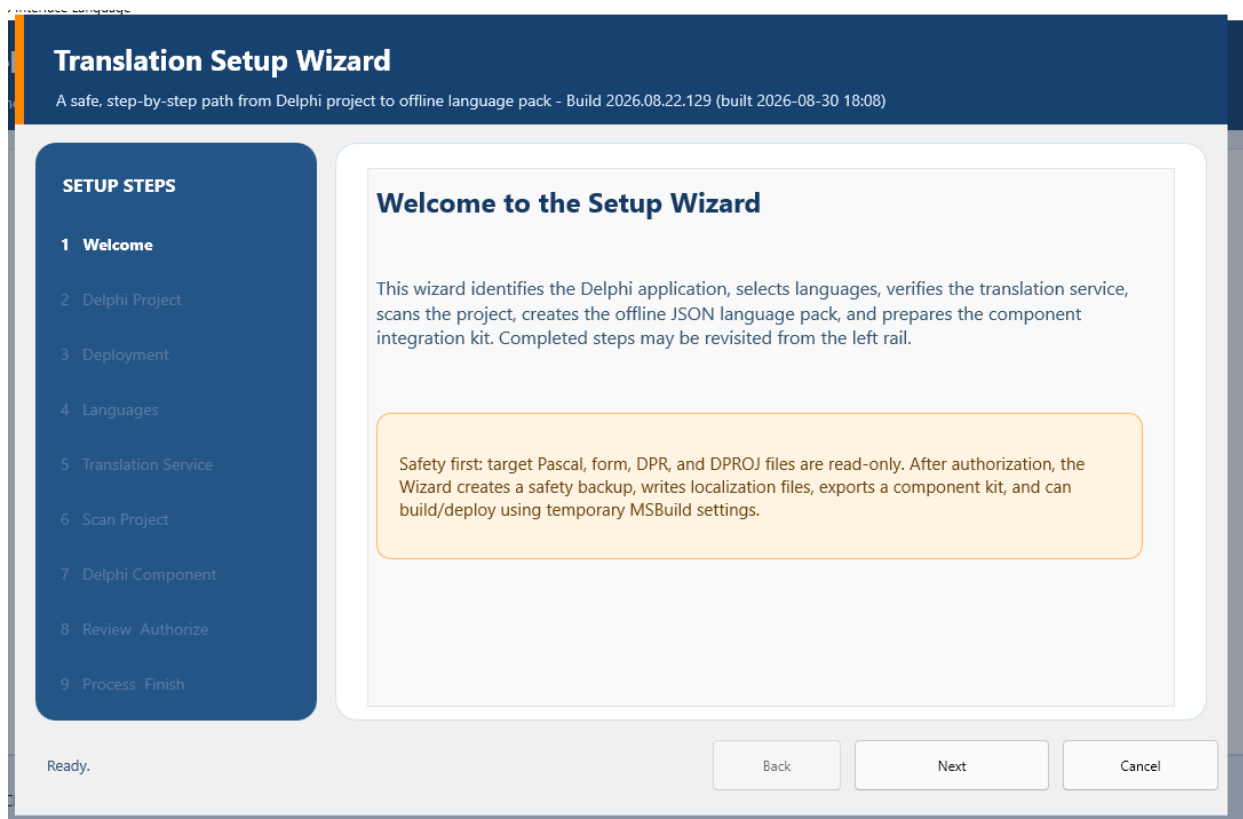


Figure 3-1. Complete Wizard frame and navigation controls.

Control or area	Purpose	Required operator action
Left step rail	Shows the eight-step sequence and the current step.	Select only a previously reached step; unreached steps remain disabled.
Next	Validates the current step and advances.	Press Enter when Next is the default button.
Back	Returns to the previous reached step.	Use before final processing; it is hidden or disabled when backward movement is unsafe or meaningless.
Cancel	Closes an ordinary Wizard session or a stopped run.	Press Esc before final processing. After successful completion use Finish instead.
Begin Final Processing	Replaces Next on Review and Authorize.	Select only after the project-closed and authorization checks are true.
Finish	Closes a successful completed run.	Press Enter after reading the completion status.
Footer status	Reports the last completed action and next requirement.	Read it whenever a button remains disabled.

Final-processing boundary. Before Begin Final Processing, Cancel leaves the target unchanged. During processing, closing is blocked while an active operation must finish or stop safely. On success Back and Cancel disappear because Finish is the only meaningful close action.

4. Step 1 - Welcome

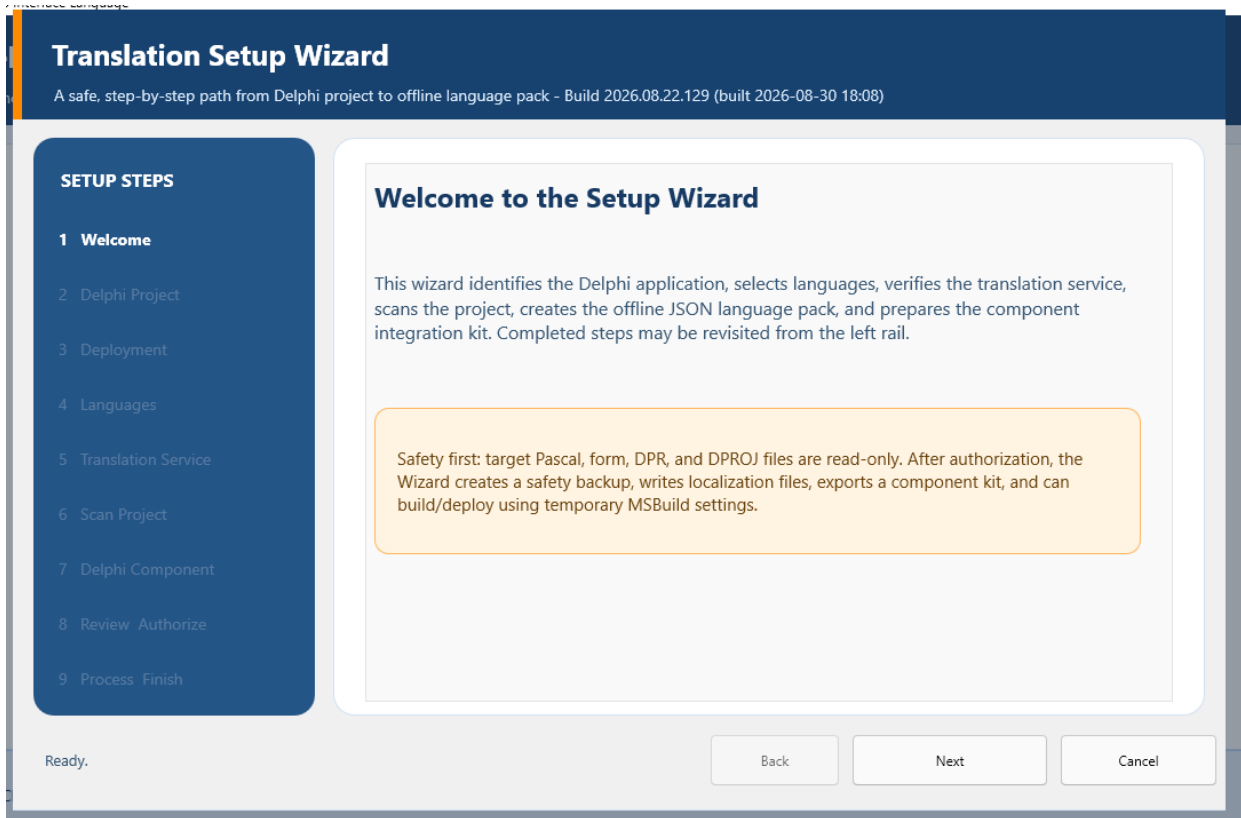


Figure 4-1. Step 1 - Welcome.

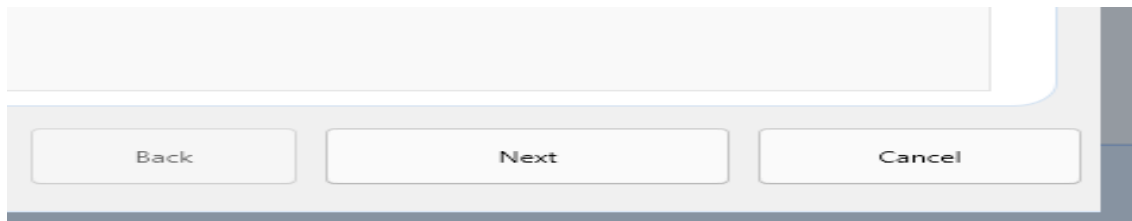


Figure 4-2. Welcome-page navigation buttons.

The Welcome page establishes what the Wizard will identify and produce. At this point no project is selected and no provider request, scan, backup, export, or deployment has started.

Read the safety notice. It explains that final processing creates a backup and Studio-owned artifacts while target Pascal, form, DPR, and DPROJ files remain read-only. Press Enter or select Next.

Control or area	Purpose	Required operator action
Welcome explanation	Summarizes project identification, languages, provider verification, scanning, packs, and component integration.	Confirm that this is the intended guided workflow.
Safety notice	Defines the read-only target-source contract and required backup.	Do not continue if the selected target copy is not disposable or protected.
Next	Opens Delphi Project.	Default action; Enter should activate it.

5. Step 2 - Delphi Project

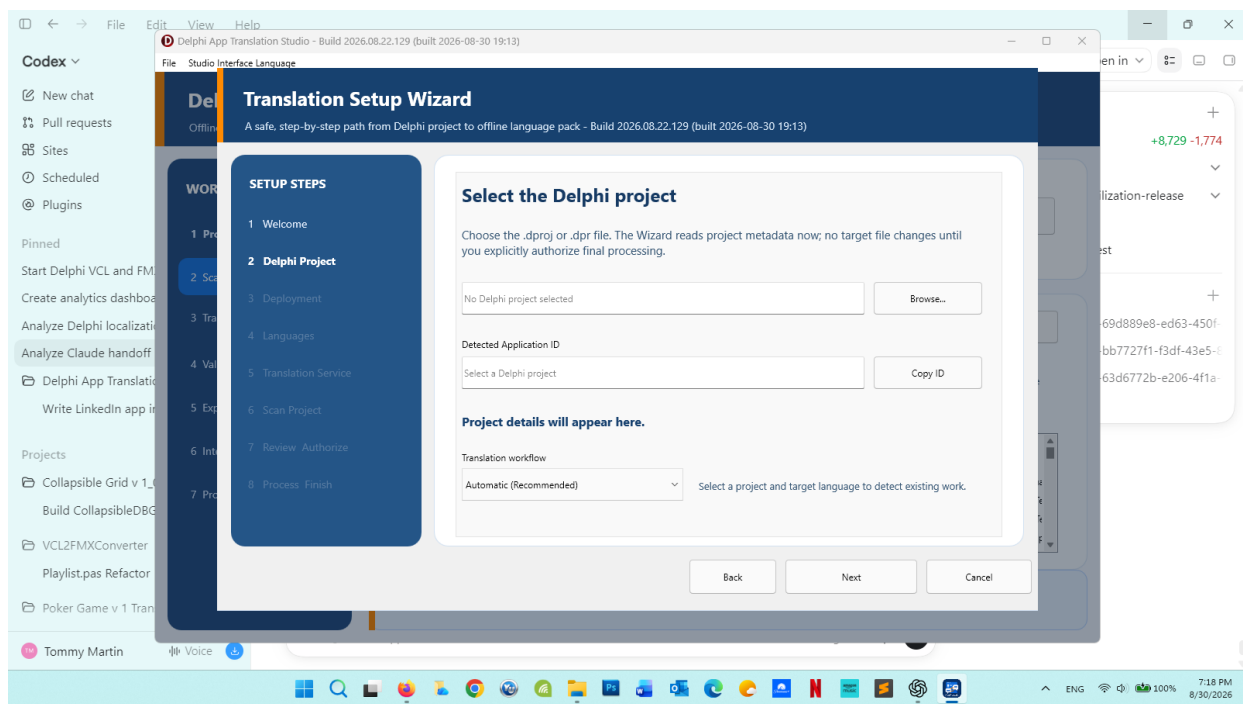


Figure 5-1. Step 2 - Delphi Project.

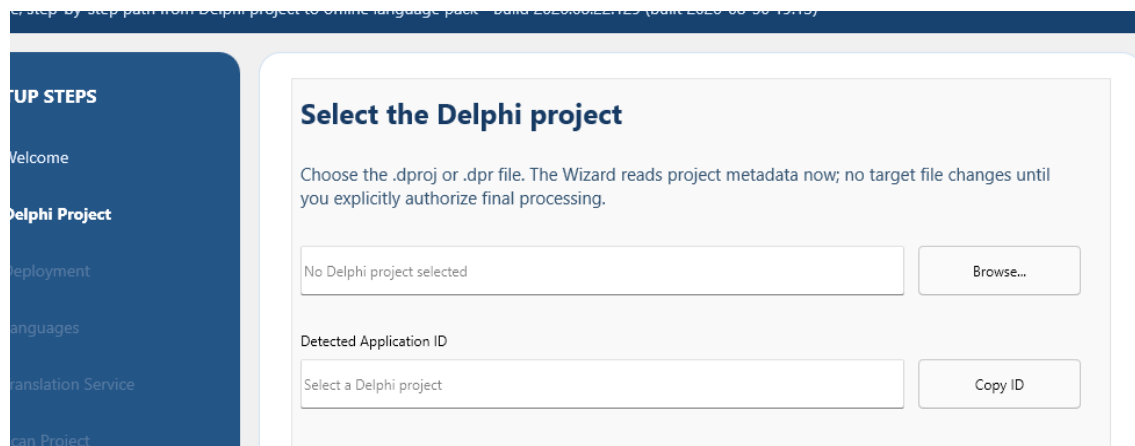


Figure 5-2. Project selection and detected identity.

Browse selects a DPR or DPROJ; DPROJ is preferred because it provides framework, platform, configuration, source, form, and output metadata. Detection is read-only and does not compile or execute the target.

ApplicationId is the stable identity that links catalog, pack, manager, workspace, and preference. It must match the manager's published ApplicationId exactly. A folder name, product caption, or executable display name is not a substitute.

Control or area	Purpose	Required operator action
Project path	Holds the selected DPR/DPROJ.	Verify the test copy path character by character.
Browse	Opens the project-file picker and reruns detection.	Choose the target DPROJ whenever available.
Application ID	Displays the detected stable application identity.	Copy it into the target manager ApplicationId property exactly.
Copy ID	Copies ApplicationId to the clipboard.	Use when configuring Object Inspector.
Framework / targets	Reports VCL or FMX and Win32/Win64 availability.	Stop if the framework is wrong or expected targets are missing.
Form resources / source units	Reports project discovery breadth.	A suspiciously low count normally means the wrong or incomplete project was selected.

Control or area	Purpose	Required operator action
Next	Accepts detected identity and opens Deployment.	Enabled only after a supported project is identified.

6. Step 3 - Deployment

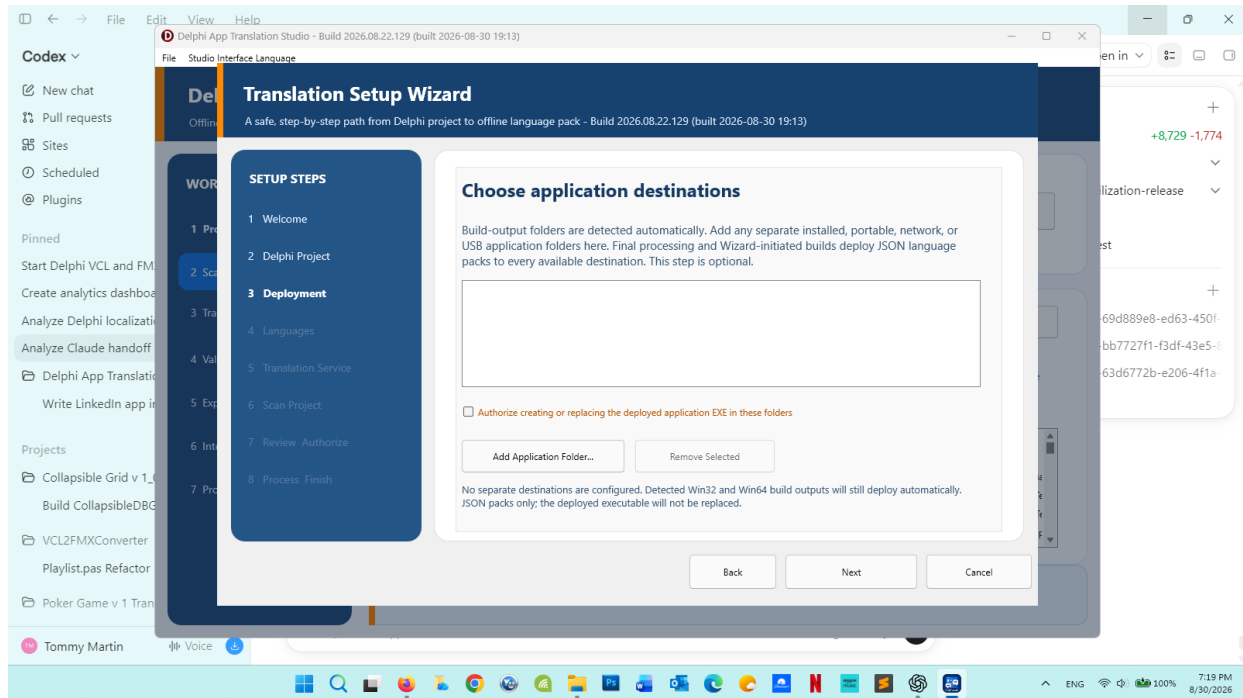


Figure 6-1. Step 3 - Deployment.

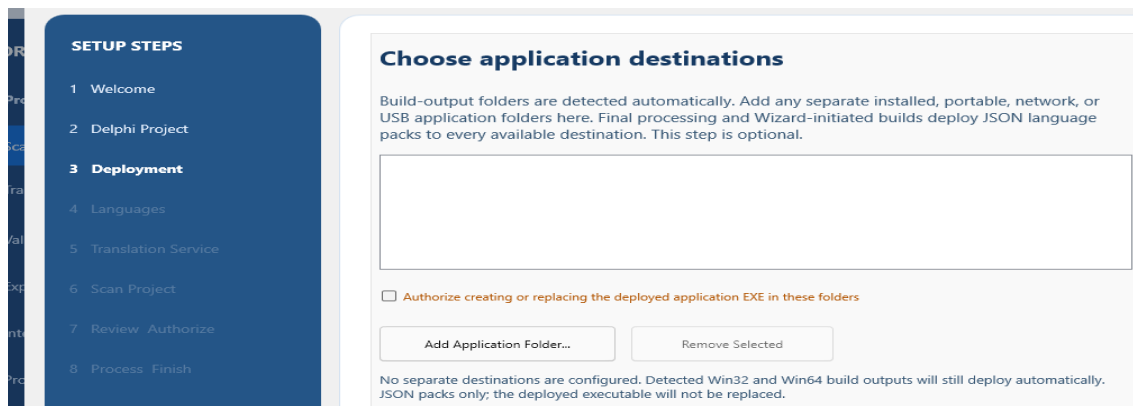


Figure 6-2. Optional destination controls.

Detected Delphi output folders require no manual entry. Add only separate installed, portable, network, USB, or test-run folders that should receive the current language packs.

Choose the application folder containing the executable, not its Localization or Languages child folder. An unavailable removable or network destination is reported and skipped; it does not invalidate the pack.

Control or area	Purpose	Required operator action
Detected outputs	Shows build-output folders inferred from the project.	Verify they correspond to the configurations you intend to run.
Destination list	Stores additional authorized application folders.	Leave empty when normal build outputs are sufficient.
Add Application Folder	Adds one executable-containing folder.	Use the full path, including drive letter.
Remove Selected	Removes an incorrect or obsolete optional destination.	Select the row first.
Deployment summary	Explains what will be deployed and when.	Read before continuing; no source file is modified.

7. Step 4 - Languages

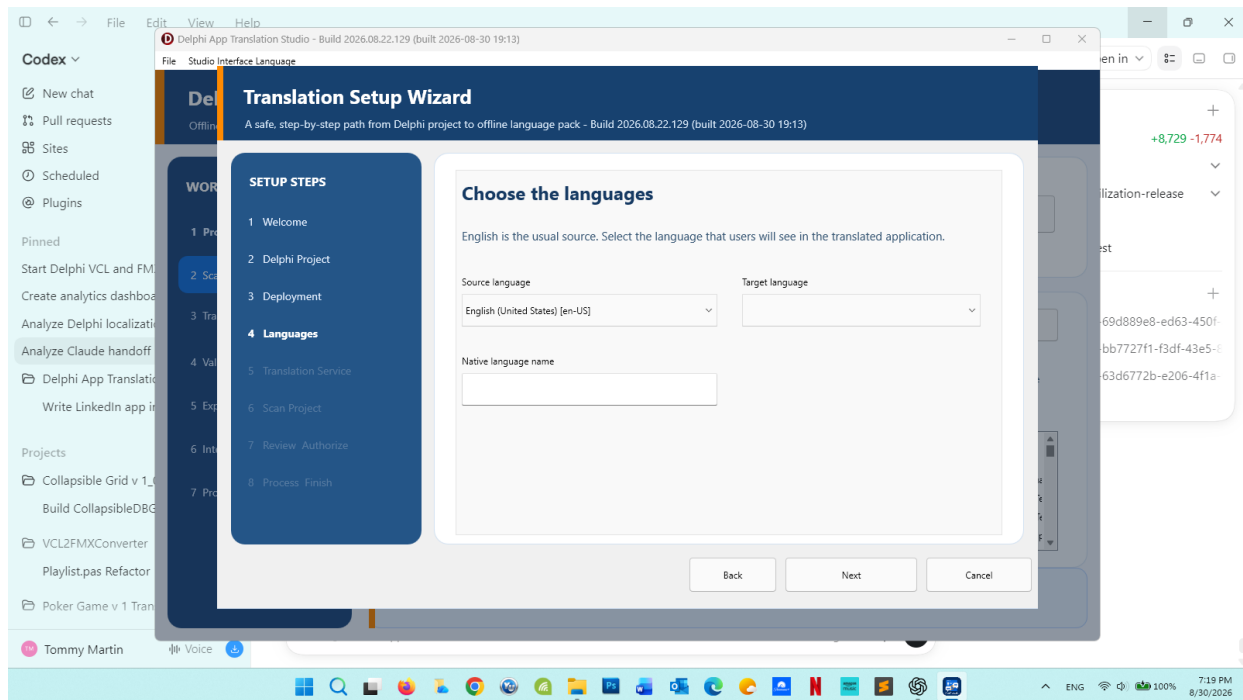


Figure 7-1. Step 4 - Languages.

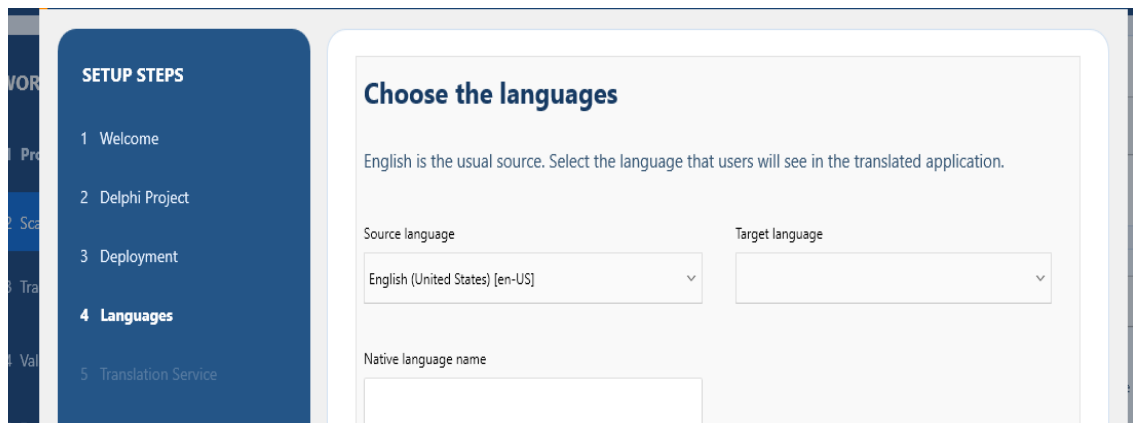


Figure 7-2. Source and target locale selection.

Source language is the language already authored in the saved forms and source. Target language is the new runtime choice. The locale code controls pack naming, native display name, direction, and formatting facts.

Changing source, target, or project invalidates downstream scan/catalog state in the current Wizard session. That reset prevents a catalog built for one identity or locale from being exported as another.

Control or area	Purpose	Required operator action
Source language	Identifies the authored source locale.	Normally English (United States) [en-US] for an English Delphi application.
Target language	Selects the pack to create or update.	Choose one locale for this Wizard run.
Native name	Shows the end-user language label.	Verify spelling and script.
Direction	Reports LTR or RTL.	Arabic, Hebrew, and Urdu must report RTL.
Locale facts	Shows date/time, separators, currency, and related regional metadata.	Review for the intended region rather than only the base language.

8. Step 5 - Translation Service

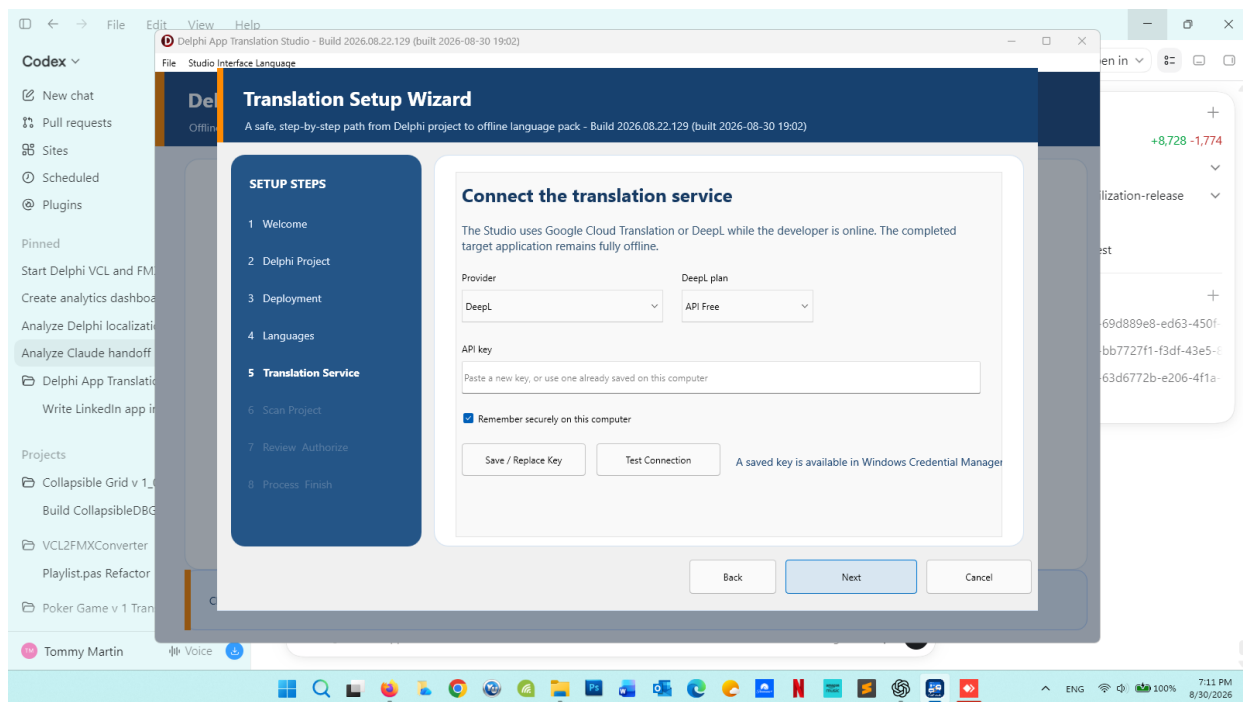


Figure 8-1. Step 5 - Translation Service.

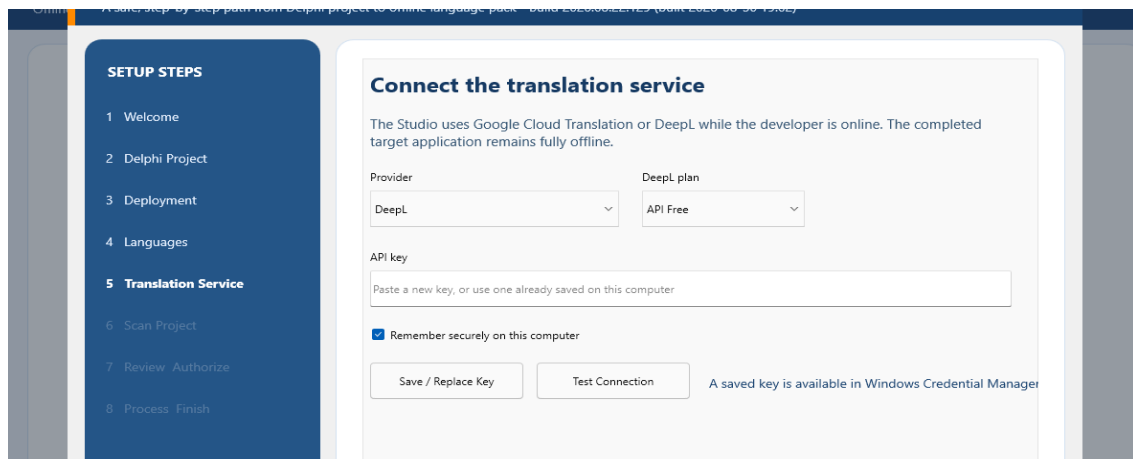


Figure 8-2. Provider, key, and connection controls.

DeepL is the first-run default and recommended provider. The Studio restores a previously saved provider/plan choice but never displays a stored secret back into the API-key edit. A blank field with a saved-key status is therefore correct.

Test Connection runs a bounded asynchronous request. The Wizard must remain responsive, cancellation-aware, and able to close after the test completes or times out. A successful test proves only authentication and endpoint access; it does not approve language quality or quota for a large catalog.

Control or area	Purpose	Required operator action
Provider	Selects DeepL or Google Cloud Translation.	Use DeepL unless project/account requirements call for Google.
DeepL plan	Selects API Free or API Pro endpoint.	Match the key's actual account plan; ignored for Google.
API key	Accepts a new or replacement secret and masks it.	Leave blank when the saved-key status says a usable credential exists.
Remember securely	Selects Windows Credential Manager instead of session-only memory.	Use only on a trusted developer computer.
Save / Replace Key	Stores provider settings and the entered secret according to the remember policy.	Use after pasting or rotating a key.

Control or area	Purpose	Required operator action
Test Connection	Performs one small provider request with timeout/cancellation.	Continue only after a success result.
Credential status	Reports saved/session availability without revealing the key.	Use it to distinguish an empty edit from a missing credential.

Secret locations. Non-secret settings are in %LOCALAPPDATA%\DelphiAppTranslationStudio\provider-settings.json. Remembered secrets use Windows Generic Credential targets DelphiAppTranslationStudio/Providers/DeepL or DelphiAppTranslationStudio/Providers/Google Cloud Translation.

9. Step 6 - Scan Project

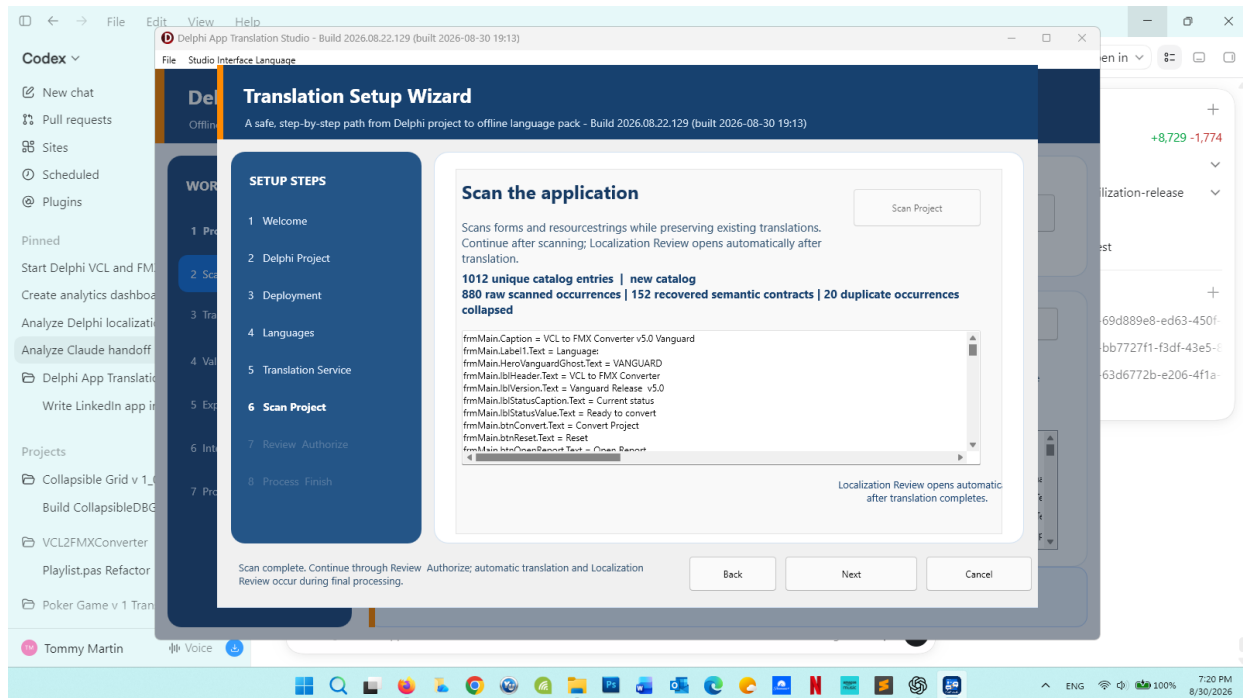


Figure 9-1. Step 6 - Scan Project.

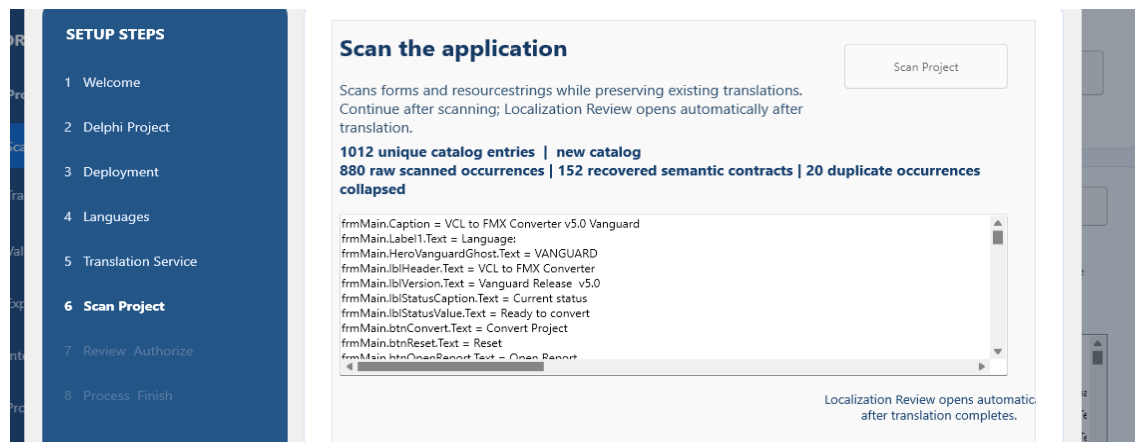


Figure 9-2. Canonical scan-count explanation.

The Wizard and Maintenance Studio call the same canonical scanner. The headline is the unique catalog-entry count. Raw observations can be lower or higher after semantic contracts are recovered and equivalent duplicate occurrences are collapsed; those supporting counts are displayed separately so the total is auditable.

The scanner reads saved text DFM/FMX resources, Pascal resourcestrings, eligible runtime assignments, and explicitly authorized project resource manifests. It does not translate arbitrary identifiers, user data, database values, or every string literal in source code.

Control or area	Purpose	Required operator action
Scan Project	Runs canonical project discovery and scan.	Use after every saved source/form text change.
Unique catalog entries	The stable-key records that enter catalog merge.	Use this as the Wizard/Maintenance comparison count.
Raw observations	Direct scanner observations before recovery/collapse.	Use to explain how the unique total was formed.
Recovered semantic contracts	Known dynamic/application text restored through stable semantic rules.	Review when runtime text coverage changes.
Duplicate occurrences collapsed	Equivalent repeated observations represented once.	A nonzero value is expected when identical ownership contracts repeat.

Control or area	Purpose	Required operator action
Scan list	Shows stable key and source text.	Inspect representative forms, components, properties, resourcestrings, and dynamic contracts.

10. Step 7 - Review and Authorize

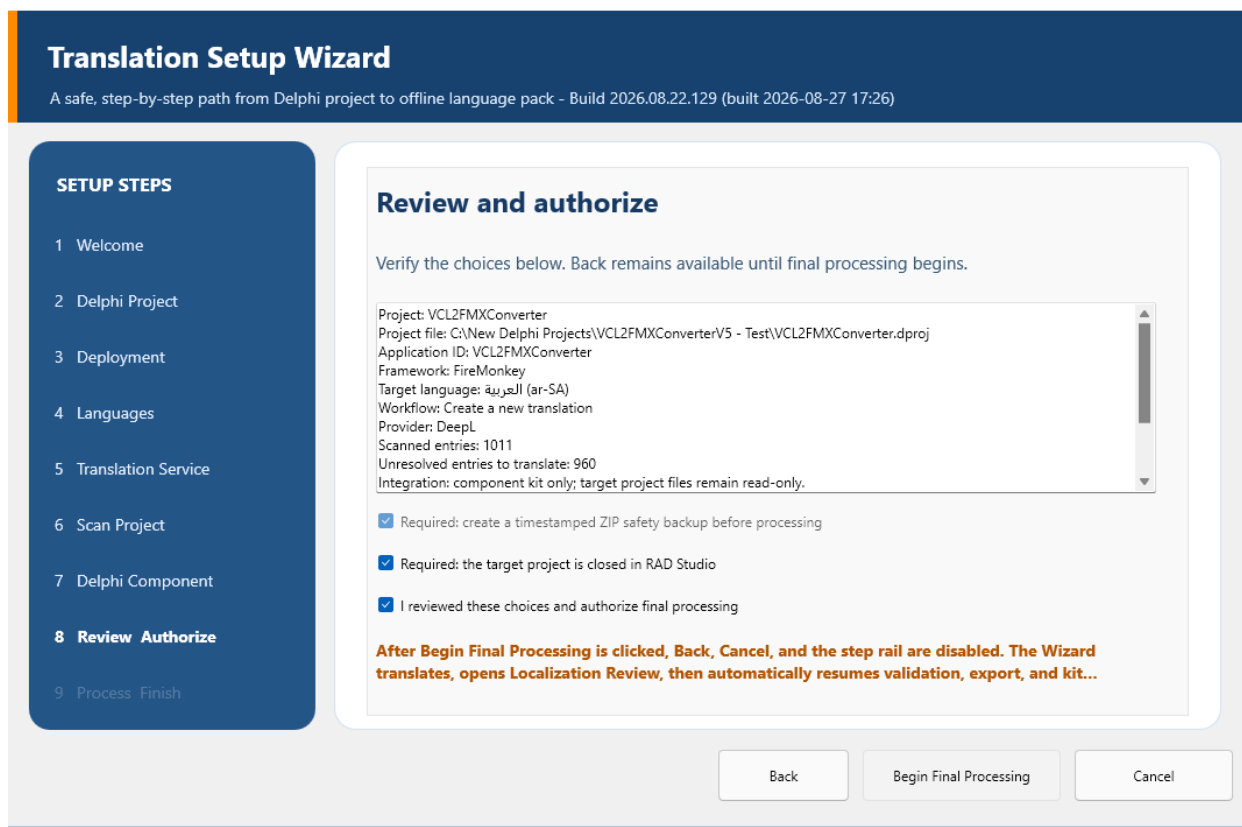


Figure 10-1. Step 7 - Review and Authorize content area.

This is the final deliberate authorization point. The summary must describe the exact test project, framework, source/target locales, provider, scan state, unresolved work, backup, and component workflow that the developer intends.

The target project must be closed in RAD Studio. An open DPROJ or form may leave unsaved buffers and makes the selected source snapshot unreliable even though the Wizard itself does not rewrite those files.

Control or area	Purpose	Required operator action
Review memo	Prints the complete proposed operation in plain text.	Read the path, application ID, locale, provider, counts, output, and backup line by line.
Project closed confirmation	Confirms RAD Studio no longer owns an in-memory target-project state.	Close the project, then select it.
Safety backup	Requires the pre-processing ZIP.	Leave enabled; final processing will not start without it.
Authorization	Records that the developer reviewed the operation.	Select only after all summary values are correct.
Begin Final Processing	Starts backup, translation, review, validation, export, kit generation, and deployment.	This is not a generic Next button; it crosses the controlled execution boundary.

11. Localization Review During Final Processing

After provider translation and before final validation/export, the Wizard opens Localization Review automatically. This modal window is where machine output, glossary candidates, application-owned strings, layout findings, and per-language proposals are reviewed while all necessary translated text is available.

Closing Localization Review does not abandon the Wizard. It saves accepted decisions and returns control to the suspended final-processing continuation, which then validates, exports, generates the kit, deploys, and writes the completion report.

Review area	Engineering meaning	Developer decision
Findings	Information, warning, and high-risk localization issues.	Resolve high-risk issues before treating the language as release-ready.
Project glossary	Approved source/target terminology and matching rules.	Add, edit, delete, and save only product-correct terms.

Review area	Engineering meaning	Developer decision
Suggestions	Translation-memory or terminology proposals with confidence/context.	Accept, approve high-confidence, or reject explicitly.
Layout proposals	Conservative width/height/font/wrap/position changes scoped to locale and control.	Accept only after inspecting current versus proposed geometry and ownership.
Code-positioned controls	Controls whose geometry is intentionally assigned in Pascal.	Leave them alone unless the application design is changed deliberately.
Review package	HTML and JSON audit artifacts for later review.	Generate/open it when a printable or browser-based audit trail is useful.
Close	Returns accepted decisions to Wizard processing.	Close only after saving the decisions that should enter the pack.

Layout decision rule. Use natural word wrapping first, then intelligent hyphenation for long unbroken words, then a modest readable font reduction only when necessary. Never shift neighboring columns or controls merely to make one translated heading fit.

12. Step 8 - Process and Finish

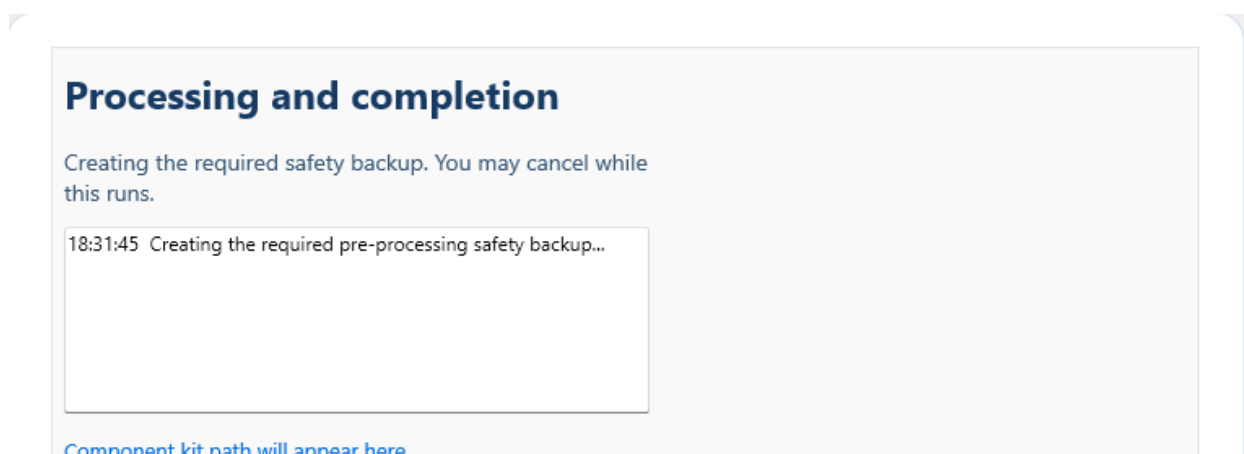


Figure 12-1. Step 8 - Process and Finish status area.

The progress memo is the authoritative sequence record. It reports backup, catalog save, provider work, review return, glossary application, validation, source/target pack export, kit generation, output deployment, optional destination deployment, and completion-report creation.

A line prefixed STOPPED is a controlled failure, not permission to ignore the problem. Read the last successful line and the stopped reason, preserve the artifacts, correct the cause, and rerun. On success the page states that target Pascal, form, DPR, and DPROJ files were not modified.

Control or area	Purpose	Required operator action
Progress memo	Timestamped operation and diagnostic history.	Read from the last line upward when a run stops.
Optional application folder	Deploys packs to one new/temporary executable folder.	Use only when the folder was not already configured in Deployment.
Rebuild before deploying	Requests a selected build before optional deployment.	Normally leave clear; select only when another compile is actually required.
Platform / configuration	Selects a supported Win32/Win64 Debug/Release build.	Do not select unsupported targets.
Deploy to Application Folders	Deploys the just-built result and packs to configured destinations.	Use when optional destination deployment is part of this run.
Finish	Closes a successful completed Wizard.	Default action after reviewing success.
Cancel after STOPPED	Closes a stopped run after diagnostics are preserved.	Use only when Finish is unavailable because processing did not complete.

13. Component Kit and dependencies Folder

The generated kit is under <Studio>\export\component-integration\<ApplicationId>. ComponentSource contains the complete current framework-appropriate unit set. The Studio does not automatically create a dependencies folder inside the target project.

A project-local dependencies folder is an optional vendoring strategy: <Target Project>\dependencies\DelphiAppTranslation\source. It makes the target build independent of the Studio repository path and lets Git record the exact runtime source shipped with that application.

1. Open <Studio>\export\component-integration\<ApplicationId>\ComponentSource.
2. Create <Target Project>\dependencies\DelphiAppTranslation\source if the project will vendor the runtime.
3. Copy every PAS unit from ComponentSource into that source folder. Do not select only the five files that happened to be named in one compiler error.
4. When the Studio/runtime changes, regenerate the kit and refresh the entire vendored set together so interfaces and implementations remain compatible.
5. Commit the dependency source with the target test project when licensing and repository policy permit. Never copy provider credentials or development catalogs with it.

Framework scope	Complete unit set
Shared by VCL and FMX	DAT.Core.AtomicFile; DAT.Core.Diagnostics; DAT.Runtime.LanguagePack; DAT.Runtime.Preference; DAT.Runtime.Manager; DAT.Runtime.LayoutOverrides; DAT.Runtime.SplashTranslation; DAT.Runtime.TemplateRewrite; DAT.Components.Core
VCL-specific additions	DAT.Runtime.VCL; DAT.Runtime.SplashTranslation.VCL; DAT.Runtime.TemplateRewrite.VCL; DAT.Components.VCL; DAT.Components.VCL.LanguageSelector
FMX-specific additions	DAT.Runtime.FMX; DAT.Runtime.SplashTranslation.FMX; DAT.Runtime.TemplateRewrite.FMX; DAT.Components.FMX; DAT.Components.FMX.LanguageSelector

14. Delphi Compiler Search Path

The Wizard passes ComponentSource temporarily to Wizard-initiated build commands but does not edit DPROJ. A manual RAD Studio build therefore needs a permanent Search path entry that resolves the current ComponentSource or vendored dependency folder.

1. Open the target project in RAD Studio and choose Project > Options.
2. At the top choose All configurations and All platforms unless the project intentionally uses distinct paths.
3. Open Building > Delphi Compiler and locate Search path.

4. Append `.\dependencies\DelphiAppTranslation\source` when using the recommended project-local folder. An absolute example is `C:\New Delphi Projects\MyApplication\dependencies\DelphiAppTranslation\source`.
5. Preserve every existing path and the inherited `$(DCC_UnitSearchPath)` value. Do not replace the field with only the DAT path.
6. Save the option, clean, and build every configuration/platform that will ship.

Verified toolchain path. RAD Studio environment: `C:\Program Files (x86)\Embarcadero\Studio\37.0\bin\rvars.bat`. Win32 compiler: `C:\Program Files (x86)\Embarcadero\Studio\37.0\bin\dcc32.exe`.

15. Files and Folders Created or Used

Path or file	Contents and reason
<code>%LOCALAPPDATA%\DelphiAppTranslationStudio\provider-settings.json</code>	Contains provider, plan, remember policy, timeout, and batch size, but never the API key. It restores non-secret Studio provider behavior.
<code>%LOCALAPPDATA%\DelphiAppTranslationStudio\language.ini</code>	Contains the Studio interface locale. It keeps the Studio UI language independent of target projects.
<code>%LOCALAPPDATA%\DelphiAppTranslationStudio\Workspaces\<ApplicationId>\Development</code>	Contains full development catalogs. It preserves stable keys, source, translations, status, context, and scan provenance.
<code>...\Languages</code>	Contains canonical source and translated runtime packs. It provides the workspace copy used for kit generation and deployment.
<code>...\Glossaries</code>	Contains project terminology JSON. It preserves approved product wording across runs.
<code>...\Deployment</code>	Contains additional application destinations. It remembers authorized pack-deployment folders.

Path or file	Contents and reason
%LOCALAPPDATA%\<ApplicationId>\language.ini	Contains the target application's selected locale unless PreferenceLocation is customized. It restores the end user's language at startup.
<Studio>\export\localization-review\<ApplicationId>\<locale>	Contains HTML/JSON review artifacts. It creates a durable linguistic and layout audit trail.
<Studio>\export\component-integration\<ApplicationId>	Contains ComponentSource, packs, README, manifest, script, and completion report. It supplies the complete target integration set without source injection.
<Target EXE>\Localization\Languages	Contains deployed runtime packs. This is the default path resolved by LanguagesFolder.

Atomic recovery files. A .tmp file is an interrupted candidate, .previous is the last known-good version, and .corrupt-**<timestamp>** is quarantined invalid content. Do not delete recovery files until the active JSON has been validated and backed up.

16. Build and Deployment Procedure

1. Confirm the complete dependency source and Search path before compiling.
2. Build Win32 Debug and Win32 Release. Build Win64 only when the target application will ship it and the target's dependencies support it.
3. Copy the canonical source pack and every translated pack to each executable's Localization\Languages folder, or use the Wizard deployment action.
4. Run the executable from the output folder you just built, not an older copy elsewhere on disk.
5. Repeat pack deployment after any catalog, glossary, layout-decision, or source-pack change.

Build	Minimum deployment check	Runtime check
Win32 Debug	EXE plus source/target packs in its output tree.	Detailed first-pass diagnostics and all language switching.
Win32 Release	Independent Release output receives identical pack set.	Release candidate behavior and startup preference.

Build	Minimum deployment check	Runtime check
Win64 Debug	Only when the application supports Win64.	Architecture-specific runtime and layout behavior.
Win64 Release	Only when it will ship.	Final Win64 release candidate.
Installed/ portable folder	Executable identity verified before copying packs.	Run in place, including removable/network availability rules.

17. Complete Runtime Verification

1. Start in canonical English/source language and inspect every form, dialog, menu, tab, grid, report, HTML page, message, and dynamic status area.
2. Switch to each translated LTR language. Confirm static controls, dynamic strings, templates, HTML/report content, accelerator keys, placeholders, and persisted data.
3. Switch between two non-English languages without returning to English. No previous-language text or image may remain.
4. For Arabic, Hebrew, and Urdu, confirm paragraph direction and control alignment are RTL while identifiers, file paths, numbers, WinAPI, FMX, and other protected LTR tokens remain readable.
5. Switch back to English. The current screen must repaint immediately; no blank content panel or delayed click is acceptable.
6. Close and restart in each representative locale. Confirm the preference loads quickly and an invalid/missing pack falls back safely to the canonical source pack.
7. Inspect long headings and paragraphs at normal DPI and any supported scaling. Use natural space wrapping, intelligent hyphenation, and only modest font fitting; reject clipping, overlap, premature wrapping, or shifted neighboring columns.
8. Repeat the complete matrix for each shipped platform/configuration and record any exception by stable key, form, component, locale, and build.

18. Updating, Resuming, and Adding Languages

A later scan merges into the same application/locale catalog. Matching stable key and source text preserve existing work; changed source becomes attention-required; new entries are added; missing entries become obsolete rather than disappearing silently.

The provider receives only eligible unresolved records. Reviewed, approved, excluded, obsolete, and unchanged translated records are not retranslated merely because the Wizard runs again.

1. Make source/form changes in the test project and choose Save All.
2. Run the Wizard with the same ApplicationId and target locale, then scan again.
3. Review new, changed, unchanged, recovered, and obsolete counts before authorization.
4. Translate unresolved work, complete Localization Review, validate, export, regenerate the kit, and redeploy.
5. Refresh the complete vendored dependency set only when runtime/component source changed; do not replace it for ordinary catalog-only updates.
6. For another language, select that target locale and repeat scan/translation/review/export. Deploy the canonical source pack and all translated packs together.

19. Failure Recovery and Troubleshooting

Symptom	Most likely cause	Required correction
Project is not recognized	Wrong/partial DPR/DPROJ or unsupported metadata.	Select the saved DPROJ from the complete test copy and verify it opens in RAD Studio.
Scan is zero or unexpectedly small	Wrong project, unsaved forms, binary/unreadable resources, or incomplete source closure.	Save All, verify text resources and detected counts, then rescan.
Wizard and Maintenance totals differ	Different project/source snapshot, old catalog merge basis, or comparing unique entries with raw observations.	Use the same project and compare the same unique-entry headline and supporting counts.
Connection test never completes	Network/provider request did not honor timeout/cancellation or an old build is running.	Run the current build, verify timeout/settings, provider endpoint, firewall, and account; do not kill processing before diagnostics.
401/403	Invalid key, wrong DeepL plan, disabled Google API, billing, or restriction problem.	Correct account/endpoint/key restrictions and retest.
429	Quota or rate limit.	Wait for the provider window, reduce work/batch pressure, or correct quota/billing.

Symptom	Most likely cause	Required correction
Selector is empty	Packs missing, invalid, wrong ApplicationId/framework/version, or wrong executable folder.	Inspect the actual running EXE folder and validate source/target pack headers.
Language switch is slow	Repeated disk discovery, unnecessary full-form work, synchronous provider/network logic, or lifecycle loop.	Profile runtime switching; it must use admitted in-memory packs and bounded open-form application only.
Mixed languages remain	Previous-language dynamic/template text was not restored before applying the new pack.	Verify source snapshot/dynamic reverse mapping and language-change refresh contracts.
English switch shows a blank page	Current page was cleared without immediate rebuild/repaint.	Verify the language-changed handler rebuilds current dynamic/HTML content synchronously.
RTL text is left aligned	A paragraph/control direction contract is absent or overridden.	Verify pack direction, framework applicator, and per-control ownership; preserve protected LTR tokens.
Compiler cannot find DAT units	ComponentSource/dependency folder is missing or Search path is wrong for the active configuration.	Copy the full unit set and correct All configurations/All platforms Search path.
STOPPED in final log	Controlled backup, translation, review, validation, export, kit, build, or deployment failure.	Preserve the log/report, correct the specific last error, and rerun; do not treat partial output as complete.

20. Security and Privacy

- Provider API keys never belong in PAS/DFM/FMX/DPR/DPROJ, JSON catalogs, runtime packs, kits, logs, screenshots, documentation, email, issue trackers, or Git.
- Remembered keys are Windows Generic Credentials for the current user; session-only keys exist only in process memory.

- Provider-settings JSON contains no secret. Review it before source distribution only to confirm it contains non-secret configuration.
- The deployed target is offline and should not contain provider client code or credentials.
- Atomic persistence and required safety ZIP protect recoverability, but Git remains the authoritative engineering history for Studio and target projects.
- A component kit contains source and language packs. Review catalog/pack licensing and proprietary text before publishing it.

21. Printable Completion Checklist

- ☐ Complete repository downloaded or cloned; Studio builds and starts.
- ☐ Pristine target backup exists; disposable test copy builds before localization.
- ☐ Correct VCL/FMX Win32 Release design BPL installed through Install Packages.
- ☐ One manager and one connected selector saved on the primary form.
- ☐ ApplicationId, LanguagesFolder, and SourceLanguage verified in Object Inspector.
- ☐ Target project saved and closed before final processing.
- ☐ DeepL/Google key stored with intended policy and Test Connection passed.
- ☐ Project, source locale, target locale, native name, direction, and locale facts verified.
- ☐ Scan headline and supporting raw/recovered/duplicate counts reviewed.
- ☐ Review and Authorize summary, safety ZIP, project-closed check, and authorization confirmed.
- ☐ Localization Review findings, glossary, suggestions, and layout decisions completed.
- ☐ Progress ends successfully; completion report and component kit exist.
- ☐ Full ComponentSource copied to optional project dependencies folder when vendoring.
- ☐ Delphi Search path includes the exact dependency source for all shipped targets.
- ☐ Canonical source pack and every target pack deployed beside each actual EXE.
- ☐ LTR, RTL, non-English-to-non-English, switch-back, restart, dynamic, HTML, and layout matrix passes.

22. Quick Reference

Purpose	Exact or relative path
Studio repository	C:\New Delphi Projects\Delphi App Translation (example)
RAD environment	C:\Program Files (x86)\Embarcadero\Studio\37.0\bin\rsvs.bat
Win32 compiler	C:\Program Files (x86)\Embarcadero\Studio\37.0\bin\dcc32.exe
Studio settings	%LOCALAPPDATA%\DelphiAppTranslationStudio
Project workspace	%LOCALAPPDATA% \DelphiAppTranslationStudio\Workspaces\<ApplicationId>
Generated kit	<Studio>\export\component-integration\<ApplicationId>
Vendored dependencies	<Target>\dependencies\DelphiAppTranslation\source
Search path entry	.\dependencies\DelphiAppTranslation\source
Runtime packs	<Target EXE>\Localization\Languages
Target preference	%LOCALAPPDATA%\<ApplicationId>\language.ini